Performance - Complexity Trade-offs in Lightweight CNN Architectures for Blood Cell Classification

Ludergnani Brenno [†], Lazzari Tommaso[‡]

***Abstract*—Deep learning methods have achieved remarkable success in medical image classification, making the development of accurate and computationally efficient models increasingly important for real-world healthcare applications.**

This work investigates the trade-off between model complexity and predictive performance for blood cell image classification using the BloodMNIST dataset. Rather than designing increasingly deep architectures, we focus on developing lightweight convolutional neural networks capable of achieving competitive results while maintaining low computational requirements. Starting from a simple two-layer baseline CNN, we progressively introduce data augmentation, batch normalization, and spatial attention mechanisms to evaluate their individual contributions to classification performance and efficiency.

Experimental results reveal that increasing architectural complexity does not necessarily translate into improved performance. The baseline two-layer CNN achieved a macro F1-score of 95.17%, while the addition of data augmentation reduced performance to 90.97%. Further introducing batch normalization and spatial attention on top of augmented architectures resulted in macro F1-scores of 94.30% and 94.32%, respectively. Conversely, removing data augmentation while introducing batch normalization and spatial attention produced the best results, achieving macro F1-scores of 96.95% and 97.13%.

These findings demonstrate that lightweight CNN architectures can provide highly competitive performance for medical image classification while maintaining low computational cost. Moreover, the experiments underline that increasing complexity in data processing techniques and models' architectures does not necesserily translate in better results and that is worth meticolously investigate each architecture's component contribution to the final performances.

***Index Terms*—Blood Cell Classification, Medical Image Classification, Convolutional Neural Networks, Lightweight Deep Learning, Performance-Efficiency Trade-offs, BloodMNIST**

I. Introduction

Deep learning has become one of the dominant paradigms in medical image analysis, enabling substantial improvements in tasks such as disease detection, image classification, and diagnostic support. Convolutional Neural Networks (CNNs), in particular, have demonstrated strong performance across numerous biomedical imaging applications, motivating their widespread adoption in healthcare-related machine learning pipelines. However, increasing model complexity often comes at the cost of higher computational requirements, creating a growing need for architectures that balance predictive performance with computational efficiency.

Despite the success of increasingly deep and complex neural architectures, designing efficient models remains an open challenge. Recent benchmark initiatives such as MedMNIST were specifically introduced to facilitate the evaluation of machine learning algorithms in biomedical image classification settings. BloodMNIST, one of the MedMNIST datasets, offers a multi-class blood cell classification task. Existing literature extensively investigates performance improvements through increasingly sophisticated architectures, yet comparatively less attention is devoted to systematically analyzing how architectural modifications affect the trade-off between predictive accuracy and computational cost.

This report investigates the trade-off between predictive performance and architecture complexity in CNN-based medical image classification using the BloodMNIST dataset. We conduct a systematic empirical study through the progressive development and evaluation of six convolutional architectures ranging from a simple baseline CNN to more sophisticated variants incorporating batch normalization, data augmentation strategies, and attention mechanisms. Rather than proposing a novel architecture, our objective is to quantify how different architectural design choices influence classification performance, model complexity, computational requirements, and inference efficiency. By jointly evaluating predictive metrics and computational indicators, this work aims to provide practical insights into how lightweight medical imaging models can be designed while maintaining competitive classification performance.

This report is structured as follows. Section II reviews the existing literature on blood cell classification, lightweight biomedical benchmarks, and convolutional architectures for medical imaging applications. Section III introduces the proposed processing pipeline and provides a high-level overview of the investigated architectures. Section IV describes the input signals, preprocessing procedures, and feature extraction strategy. Section V details the proposed learning framework, including architectural components, optimization procedures, and training strategies. Section VI presents the experimental evaluation and analyzes predictive performance, computational efficiency, and class-wise behaviors. Finally, Section VII concludes the report and discusses limitations, practical implications, and possible future research directions.

[†]Department of Mathematics, University of Padova, email: brenno.ludergnani@studenti.unipd.it

[‡]Department of Mathematics, University of Padova, email: tommaso.lazzari.1@studenti.unipd.it

II. Related Work

Deep learning has been widely investigated for microscopic blood cell image classification. Acevedo et al. introduced the Peripheral Blood Cell (PBC) dataset, a publicly available collection of 17,092 expert-annotated images of normal peripheral blood cells organized into eight classes [1]. The dataset was designed to support the development and benchmarking of automatic recognition systems and has become an important reference for blood cell classification studies. However, the original dataset contribution mainly focuses on data availability and benchmark creation rather than on the systematic evaluation of lightweight architectures. Our work builds on this line of research by using BloodMNIST, a standardized derivative of this dataset, to analyze how compact CNN models behave under different architectural design choices.

Yang et al. proposed MedMNIST v2, a large-scale benchmark collection for lightweight 2D and 3D biomedical image classification [2]. Their work is particularly relevant because it standardizes multiple biomedical datasets into small image formats and provides reproducible train-validation-test splits. BloodMNIST is included as an eight-class blood cell microscopy task, making it suitable for fast experimentation and fair comparison of machine learning methods. Nevertheless, MedMNIST mainly provides benchmark baselines and broad dataset coverage, while it does not specifically investigate the internal effect of common CNN components such as data augmentation, batch normalization, and attention mechanisms on the performance-efficiency trade-off. Our work extends this perspective by focusing on a controlled ablation-style study over a family of lightweight CNNs.

Liu et al. developed AIMIC, a code-free deep learning platform for microscopic image classification, and evaluated several state-of-the-art CNN architectures on multiple blood cell datasets [3]. Their results show that deeper architectures such as ResNeXt and ResNet can achieve strong classification performance, while lightweight models such as MobileNetV2 may offer a better balance between accuracy and inference cost. This work is important because it explicitly considers computational efficiency, not only predictive performance. However, the analysis mainly compares existing large-scale and mobile-oriented architectures rather than studying how progressively modifying a simple CNN affects both accuracy and computational cost. In contrast, our report starts from a minimal two-layer CNN and incrementally adds architectural components to isolate their practical impact.

Overall, previous studies demonstrate the effectiveness of CNN-based methods for blood cell classification and provide valuable benchmark datasets and reference models. However, less attention has been devoted to understanding whether additional architectural complexity is always beneficial in lightweight medical image classification.

III. Processing Pipeline

The proposed framework follows a structured processing pipeline designed to systematically evaluate the relationship between architectural complexity and predictive performance in blood cell classification. The overall workflow consists of three main stages: data preprocessing, convolutional neural network processing, and performance evaluation.

A. Data Preprocessing Pipeline

The experimental pipeline starts from the BloodMNIST dataset, composed of RGB microscopic blood cell images belonging to eight different categories. Images are resized to a fixed resolution of 64×64 pixels and normalized before training to ensure stable optimization and comparable feature distributions across splits. More specifically, pixel intensities are first scaled into the [0,1] interval and subsequently standardized using channel-wise statistics computed exclusively on the training set, which are then applied to validation and testing samples to avoid information leakage. Dataset samples are finally organized into TensorFlow data pipelines using shuffling, batching, and prefetching operations to improve training efficiency and hardware utilization.

B. Progressive CNN Architecture Design

The core idea behind the proposed framework is to progressively increase architectural complexity while maintaining compact network structures. The starting point is a lightweight baseline CNN (Model A) composed of two convolutional blocks followed by a small fully connected classifier. Additional models are obtained through incremental modifications to this baseline architecture.

The first family of architectures investigates the effect of introducing data augmentation during training, followed by batch normalization and spatial attention mechanisms (Models B,C,D). A second family removes augmentation while preserving normalization and attention components (Models E,F). This progressive design enables controlled comparisons between architectures that differ by only a small number of processing blocks, allowing the contribution of individual components to be isolated.

Across all architectures, feature extraction is performed using two convolutional stages followed by spatial dimensionality reduction through max-pooling operations. The resulting feature maps are transformed into compact vector representations and processed through dense classification layers that produce the final probability distribution over the eight blood cell categories. Architectural complexity is intentionally constrained, with all investigated models remaining close to approximately 2.1 million parameters despite increasing functional complexity.

C. Training and Evaluation Pipeline

All models are trained using identical optimization procedures to ensure fair comparisons between architectures. Training employs mini-batch optimization with early stopping and adaptive learning rate reduction to avoid overfitting and unnecessary computational cost. Training metadata, including parameter counts, training duration, and model size, are

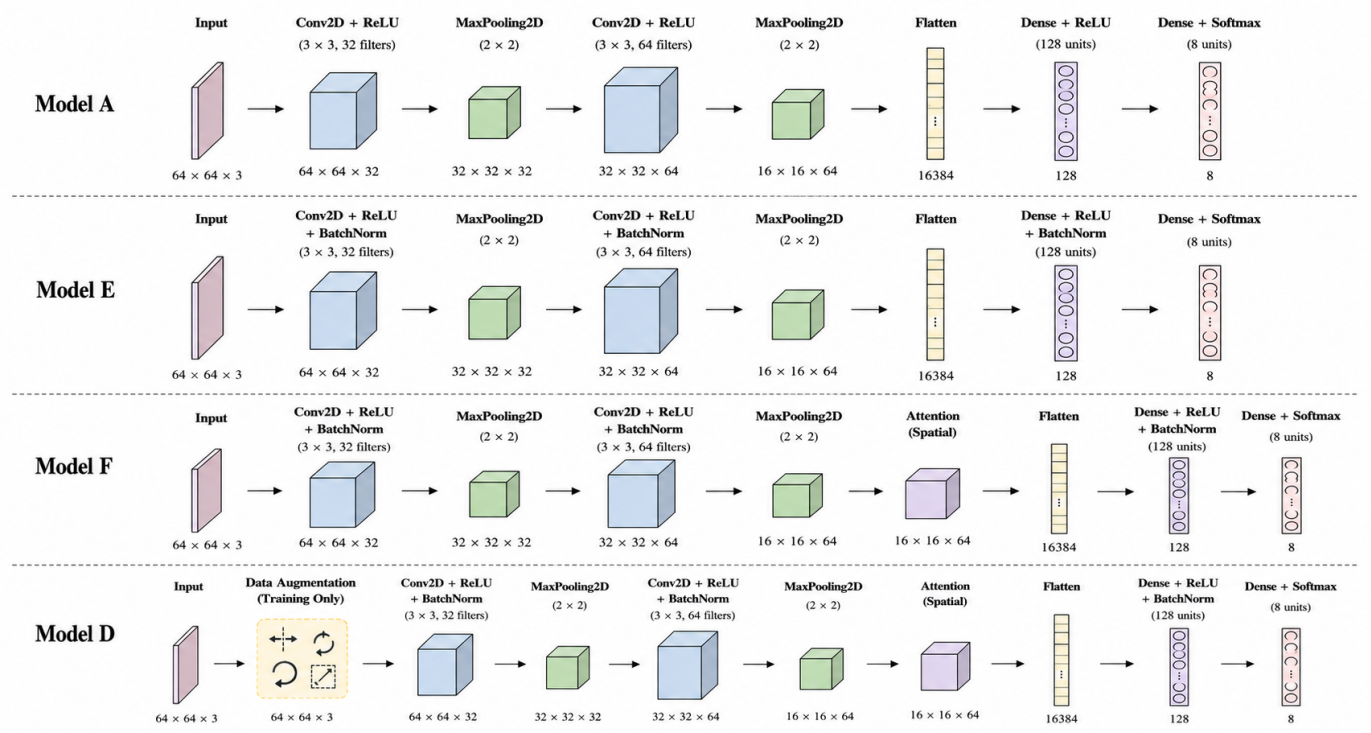


Fig. 1: Architectural overview of the investigated CNN models. Starting from a lightweight baseline architecture (Model A), additional processing blocks including data augmentation, batch normalization, and spatial attention are progressively introduced to study their impact on predictive performance and computational efficiency.

recorded automatically during execution. Evaluation is subsequently performed on the independent test partition using predictive metrics and computational indicators.

Performance assessment combines both classification quality and computational efficiency. Predictive evaluation includes macro F1-score, macro precision, macro recall, accuracy, and macro-AUC to account for class imbalance and multi-class discrimination capabilities. Computational evaluation additionally measures parameter counts, model storage requirements, training time, and inference latency. This combined evaluation strategy enables a direct analysis of the trade-off between predictive performance and computational efficiency, which represents the primary objective of this work. Fig. 1 illustrates the progression of the investigated architectures and highlights how additional processing blocks are incrementally introduced while preserving the overall lightweight design philosophy.

IV. Signals and Features

A. Input Data Description

The proposed framework operates on microscopic blood cell images obtained from the BloodMNIST benchmark dataset. BloodMNIST is a multi-class image classification dataset derived from the publicly available peripheral blood cell dataset and consists of RGB microscopic images belonging to eight blood cell categories. In this work, each image represents the input signal processed by the classification pipeline.

The dataset provides predefined training, validation, and testing partitions, enabling standardized experimental comparisons. Following the official dataset configuration, the training set contains 11,959 images, the validation set contains 1,712 images, and the test set contains 3,421 images. Each sample is represented as a three-dimensional RGB tensor of size $64 \times 64 \times 3$, corresponding to width, height, and color channels, respectively. These fixed-size image tensors constitute the direct input of the subsequent convolutional architectures.

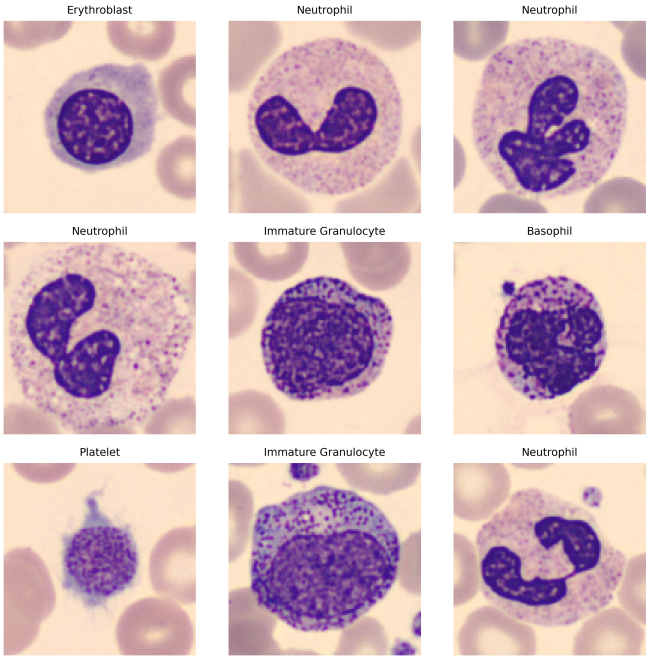
B. Data Preprocessing

Before being processed by the neural network architectures, all images undergo a standardized preprocessing pipeline designed to improve numerical stability and guarantee consistent feature distributions across dataset splits.

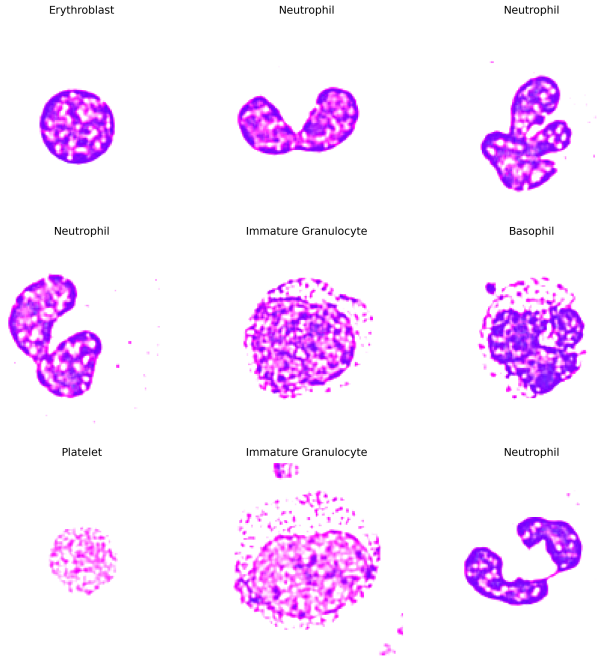
Initially, pixel intensities are converted into floating-point representations and normalized into the [0,1] interval through division by 255. Subsequently, channel-wise normalization is performed using statistics computed exclusively from the training partition. More specifically, the mean pixel values computed across the training images are equal to:

$$\mu = [0.3425; 0.3425; 0.3425]$$

while the corresponding channel-wise standard deviations are:



(a) Random samples extracted from the original BloodMNIST dataset before preprocessing.



(b) The same samples after scaling into the $[0, 1]$ interval and channel-wise normalization using training statistics.

Fig. 2: Visualization of the preprocessing pipeline applied to BloodMNIST images. The left figure shows raw dataset samples, while the right figure illustrates the effect of normalization applied using channel-wise statistics computed exclusively from the training partition. Preprocessing preserves morphological structures while transforming feature distributions into a more suitable representation for neural network optimization.

$$\sigma = [0.4138; 0.4067; 0.2963]$$

These statistics are then used to standardize validation and testing samples according to:

$$x_{norm} = \frac{x - \mu}{\sigma}$$

The result of this process is shown in Fig. 2

This strategy prevents information leakage from validation and testing partitions while ensuring that all datasets share identical normalization statistics. After normalization, the dataset is transformed into TensorFlow data pipelines using randomized shuffling, mini-batching, and prefetching operations to improve hardware utilization and training throughput. Training batches are generated using a batch size equal to 64 samples.

C. Feature Representation

Unlike traditional machine learning pipelines that rely on manually engineered descriptors, the proposed framework adopts automatic feature learning through convolutional neural networks. Consequently, no handcrafted feature extraction stage is explicitly introduced.

Feature extraction is instead performed internally by the convolutional layers, which progressively transform low-level spatial information into increasingly discriminative representations through hierarchical processing stages. Convolutional filters initially capture simple local structures such as edges, color transitions, and texture patterns, while deeper layers combine these primitive representations into higher-level semantic descriptors useful for blood cell discrimination.

The output feature maps produced by the convolutional backbone are subsequently flattened and transformed into compact vector representations that are processed by the final classification layers. This learned feature extraction strategy allows all investigated architectures to adapt feature representations directly from data rather than relying on predefined image descriptors. The only exception is represented by architectures incorporating spatial attention, where learned feature maps are additionally refined through attention-based weighting mechanisms before classification.

V. Learning Framework

This section describes the learning framework adopted to train and evaluate the proposed convolutional architectures.

A. Learning Objective

The blood cell classification problem is formulated as a supervised multi-class learning task involving eight output categories. Given an input image $x \in \mathbb{R}^{64\times64\times3}$, the objective of the learning framework is to estimate the probability distribution over the possible blood cell classes.

The neural networks produce an output probability vector through a final softmax activation layer:

$$\hat{y} = Softmax(z)$$

where z represents the logits produced by the final dense layer. Training is performed by minimizing sparse categorical cross-entropy loss:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

where y_i denotes the ground truth class label and $\hat{y}_i$ represents the predicted probability associated with the correct class.

B. Convolutional Learning Architecture

All investigated models share the same fundamental learning backbone composed of two convolutional feature extraction stages followed by dense classification layers. The baseline architecture consists of two convolutional layers with respectively 32 and 64 filters, each followed by spatial downsampling operations through max pooling. Feature maps extracted by the convolutional backbone are subsequently flattened and processed through a fully connected layer composed of 128 neurons before final classification. The baseline architecture contains approximately 2.1 million trainable parameters while remaining computationally lightweight. Starting from this baseline, progressively more sophisticated architectures are generated through incremental modifications:

- Models B,C,D investigate the effect of progressively introducing training-time data augmentation, batch normalization, and spatial attention mechanisms.
- Models E,F isolate the contribution of batch normalization and attention mechanisms without introducing augmentation operations.

This design enables controlled comparisons between architectures that differ only through specific learning components, allowing the contribution of each modification to be analyzed independently.

C. Architectural Components

The proposed architectures progressively introduce additional learning components to investigate how increasing complexity influences both predictive performance and computational efficiency.

1) Data Augmentation: Data augmentation is introduced in Models B,C and D as an implicit regularization mechanism intended to improve generalization by increasing input variability during training. Augmentation operations are applied online and exclusively during training through random horizontal/vertical flips, small rotations, and zoom transformations. More formally, given an input image x, augmentation can be represented as:

$$x_{aug} = T(x)$$

where $T(\cdot)$ denotes a stochastic transformation sampled from a predefined set of augmentation operators.

The objective of augmentation is to approximate a larger training distribution while preserving semantic consistency. Since augmentation is applied dynamically, multiple transformed versions of the same image may be observed during optimization.

2) Batch Normalization: Batch normalization is introduced to improve optimization stability and accelerate convergence by normalizing intermediate feature distributions. For a mini-batch $B = x_1, ..., x_m$, normalization is performed as:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

where ϵ is a small constant introduced for numerical stability.

The normalized activations are then rescaled and shifted through learnable parameters:

$$y_i = \gamma \hat{x}_i + \beta$$

where γ and β are trainable parameters. Batch normalization layers are inserted before activation functions throughout the convolutional backbone and dense classifier layers.

3) Spatial Attention: Spatial attention mechanisms are introduced in Models D and F to refine feature maps by emphasizing informative spatial regions while suppressing less relevant activations.

Given an intermediate feature tensor F, channel-wise average pooling and max pooling are first computed:

$$F_{avg} = AvgPool(F)$$

$$F_{max} = MaxPool(F)$$

The resulting descriptors are concatenated and processed through a convolutional layer to generate an attention map:

$$M_s(F) = \sigma(Conv([F_{avg}; F_{max}]))$$

where $\sigma(\cdot)$ denotes sigmoid activation.

Finally, the attention map is applied element-wise to the original features:

$$F' = M_s(F) \odot F$$

where $\odot$ denotes element-wise multiplication.

This mechanism allows the network to dynamically emphasize spatial locations considered more informative for blood cell discrimination before classification. Spatial attention is

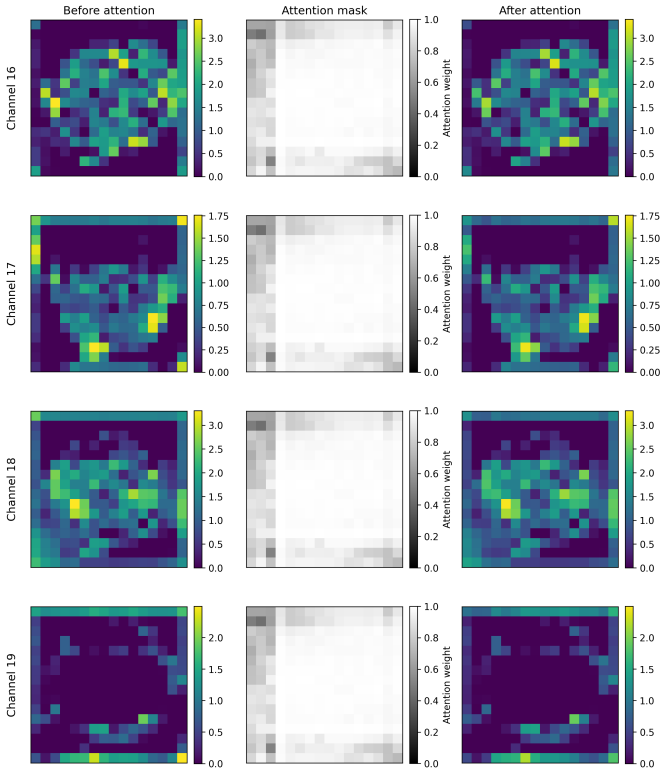


Fig. 3: Visualization of the spatial attention mechanism in Model F. The figure illustrates selected channels of the activation tensor produced after the second convolutional block for the first sample of the test set. For each channel, the left column shows the original feature activations before attention, the central column reports the learned spatial attention mask, and the right column depicts the refined activations obtained after element-wise multiplication with the attention map. Channels 16–19 are reported as representative examples.

applied after the second convolutional stage and before flattening operations.

D. Optimization Strategy

Parameter optimization is performed using the Adaptive Moment Estimation (Adam) optimizer with an initial learning rate equal to:

$$\eta = 10^{-3}$$

Adam is selected because it combines the advantages of momentum-based optimization and adaptive learning rate methods, allowing efficient optimization in high-dimensional parameter spaces while reducing sensitivity to gradient scaling. In contrast to conventional stochastic gradient descent, Adam maintains exponentially decaying estimates of both the first and second moments of the gradients.

Given the gradient g_t computed at iteration t, Adam first estimates the exponentially weighted moving averages of the gradient and squared gradient:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

where m_t represents the first moment estimate (momentum term) and v_t denotes the second moment estimate (adaptive variance estimate). Since both estimates are initialized at zero, bias correction is performed through:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

The network parameters are subsequently updated according to:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where ϵ is a small constant introduced for numerical stability.

This optimization strategy allows learning rates to adapt dynamically for each parameter, resulting in faster convergence and more stable training behavior, particularly when optimizing convolutional architectures with heterogeneous parameter distributions.

Mini-batch optimization is employed with a batch size of 64 samples. To improve reproducibility, deterministic random seeds are enforced throughout the experimental pipeline. During training, model parameters are updated iteratively through gradient-based optimization using backpropagation. Training is performed for a maximum of 50 epochs, although adaptive stopping criteria may terminate optimization earlier.

E. Training Stabilization and Regularization

Two complementary mechanisms are introduced to improve training stability and reduce unnecessary computational cost.

First, Early Stopping monitors validation loss and interrupts optimization when no improvement is observed for eight consecutive epochs, restoring the best-performing weights automatically.

Second, adaptive learning rate scheduling is implemented through a Reduce-On-Plateau strategy. Whenever validation performance stagnates, the learning rate is multiplied by a factor of 0.2 until reaching a minimum threshold of 10^{-6}.

These mechanisms allow the learning process to dynamically adapt to different architectures while maintaining identical training conditions across all experiments, ensuring fair comparisons between models.

VI. Results

A. Experimental Setup and Evaluation Metrics

Models' performance is evaluated on the independent test partition containing 3,421 images.

Since BloodMNIST exhibits class imbalance across blood cell categories, evaluation primarily focuses on macro-averaged metrics rather than accuracy alone. In particular, macro F1-score is selected as the primary evaluation metric

TABLE 1: Predictive performance comparison across architectures

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Macro AUC
A	95.47	95.67	94.77	95.17	0.9963
B	91.96	92.03	91.27	90.97	0.9952
C	94.86	94.02	95.34	94.30	0.9979
D	95.18	94.54	94.49	94.32	0.9980
E	96.90	97.08	96.83	96.95	0.9985
F	**97.14**	**97.26**	**96.99**	**97.13**	**0.9987**
ResNet50	92.08	92.23	91.65	91.92	0.9942

since it assigns equal importance to each class independently of class frequency.

Performance evaluation considers both predictive capability and computational efficiency. This combined evaluation enables direct analysis of performance-efficiency trade-offs.

B. Overall Performance Comparison

Table 1 summarizes predictive performance across all investigated architectures.

Several observations emerge from these results.

First, even with shallow and simple networks it is possible to achieve competitive results. A 2-layers CNN with batch normalization and spatial attention mechanisms achieves a F1-score of 97.13% on the BloodMNIST dataset with 64×64 image resolution.

Second, greater complexity in data processing or model architecture does not necessarily improve its performance. In fact, in these experiments augmented models (B, C and D) were outperformed by their non-augmented corresponding models.

Third, the progressive addition of both batch normalization and spatial attention mechanisms improve the performance without increasing storage requirements.

C. Computational Efficiency Analysis

Since the objective of this work is not only predictive performance but also computational efficiency, Table 2 reports complexity-related metrics.

Results indicate that architectural modifications produce only marginal increases in parameter count and storage requirements. All models remain close to approximately 2.1 million parameters and roughly 24 MB.

However, training complexity increases substantially. The introduction of batch normalization and attention mechanisms nearly quadruples training time in Models D and F relative to the baseline.

These findings suggest that predictive gains are primarily determined by architectural design choices rather than by increased parameter counts.

D. Class-wise Analysis

Confusion matrices provide additional insights into class-specific behaviors.

The majority of classes are classified with extremely high reliability across all models, particularly Eosinophils and Platelets, which frequently achieve classification accuracies close to 100%.

The most challenging category is consistently represented by Immature Granulocytes. Across multiple architectures, particularly the augmented ones, this class exhibits confusion primarily with Basophils, Monocytes and Neutrophils. This behavior likely originates from morphological similarities between these cell populations and remains visible even in the best-performing architectures.

Interestingly, the removal of augmentation substantially reduces these errors. Comparing Models B and C against Models E and F reveals significantly improved discrimination of Immature Granulocytes, supporting the hypothesis that augmentation introduces unnecessary variability for this particular classification task.

Fig. 4 illustrates representative confusion matrices for selected architectures.

VII. Concluding Remarks

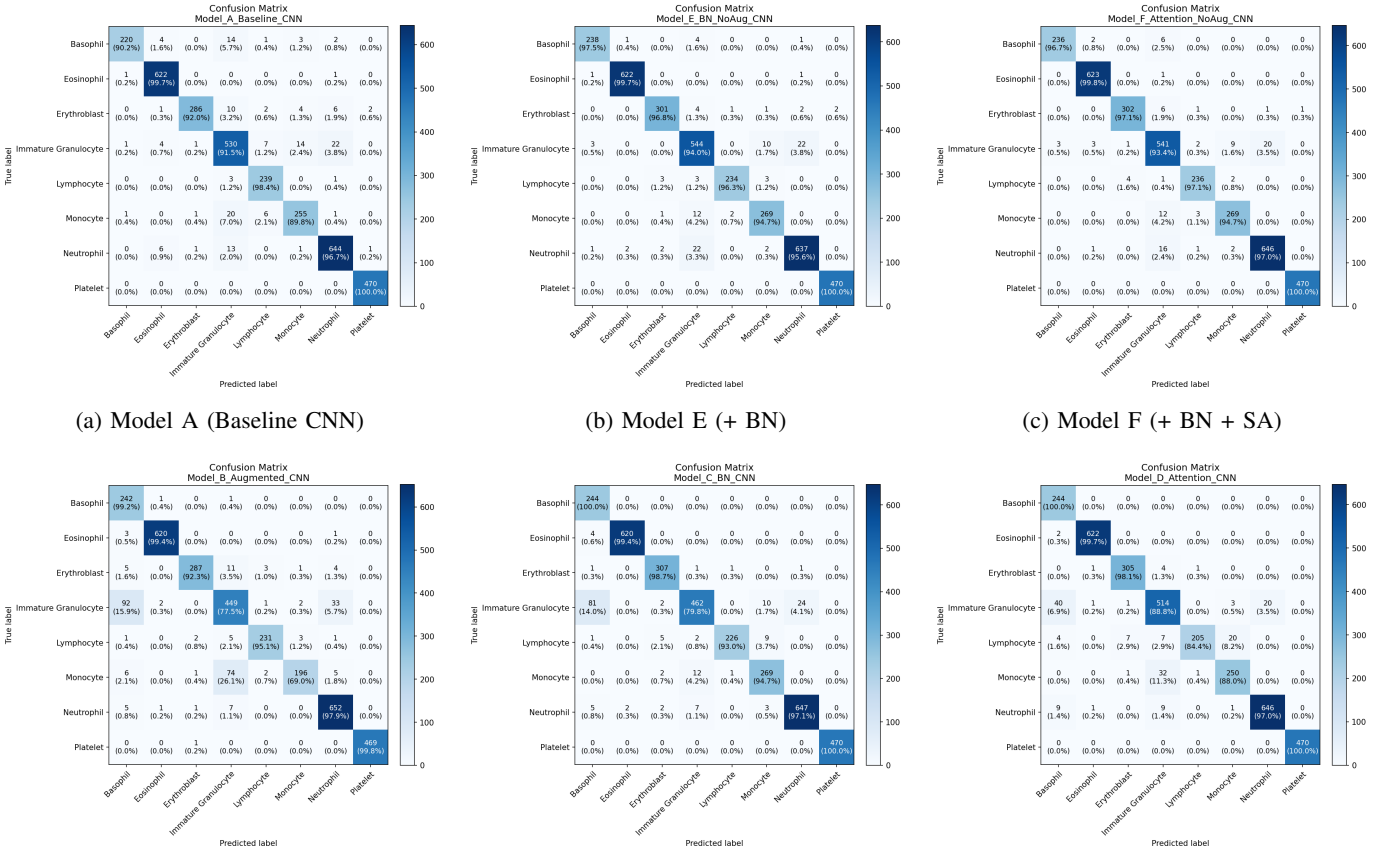
This report investigated the trade-off between predictive performance and computational efficiency in lightweight convolutional neural networks for blood cell classification using the BloodMNIST dataset. Starting from a simple two-layer CNN, progressively more complex architectures were developed by introducing data augmentation, batch normalization, and spatial attention mechanisms in order to quantify their individual contributions within a controlled experimental setting.

The experimental results demonstrated that increasing architectural complexity does not necessarily produce better predictive performance. While the baseline architecture already achieved strong classification results, the introduction of data augmentation unexpectedly degraded performance, whereas batch normalization and attention mechanisms produced significant improvements only when introduced without augmentation. The best-performing configuration achieved a macro F1-score of 97.13% while maintaining approximately 2.1 million trainable parameters and low inference latency, suggesting that compact CNN architectures can represent a practical solution for efficient medical image classification.

These findings suggest that lightweight architectures may provide an attractive alternative for medical imaging applications where computational resources are limited or where fast inference is required. More importantly, the results highlight

TABLE 2: Computational comparison across architectures

Model	Parameters	Size (MB)	Train Time (s)	Batched Latency (ms)	Unbatched Latency (ms)
A	2.118M	24.27	323	0.335	77.5
B	2.118M	24.28	350	0.319	76.6
C	2.118M	24.31	923	0.569	82.0
D	2.118M	24.32	1256	0.548	76.3
E	2.118M	24.30	978	0.512	77.0
F	2.118M	24.32	1395	0.535	77.2
ResNet50	24.144M	96.64	2983	4.033	91.7



(a) Model A (Baseline CNN)

(b) Model E (+ BN)

(c) Model F (+ BN + SA)

(d) Model B (+ AUG)

(e) Model C (+ AUG + BN)

(f) Model D (+ AUG + BN + SA)

Fig. 4: Representative confusion matrices of selected architectures. Matrices are arrenged so that in the first row are represented non augmented models' performance and in the second row augmented models' performance. Darker diagonal regions indicate stronger class discrimination performance.

the importance of carefully evaluating architectural modifications rather than assuming that additional complexity automatically leads to improved performance.

Despite these encouraging results, several limitations remain. The experimental analysis was restricted to a single dataset and focused exclusively on relatively shallow architectures. Future work could investigate whether the observed behaviors generalize to other medical imaging benchmarks, different augmentation strategies, alternative attention mechanisms, or more sophisticated lightweight architectures.

From an educational perspective, this project provided valuable insights into the complexity of designing reliable experimental frameworks for neural network comparison. One of the most important lessons learned was that obtaining meaningful comparisons between architectures requires careful control of preprocessing pipelines, optimization procedures, evaluation metrics, and computational measurements.

One of the main difficulties encountered during development originated from the initial project direction. The original objective was to design a custom high-performing neural network specifically optimized for this task. However, preliminary experiments quickly revealed that achieving strong predictive performance on BloodMNIST was relatively straightforward. Consequently, the project direction was re-

considered and shifted toward a more systematic analysis of how individual architectural components influence performance within lightweight convolutional networks. This change ultimately resulted in a more informative investigation of performance-complexity trade-offs rather than a simple architecture optimization exercise.

REFERENCES

[1] A. Acevedo, A. Merino, S. Alférez, Á. Molina, L. Boldú, and J. Rodellar, "A dataset of microscopic peripheral blood cell images for development of automatic recognition systems," *Data in Brief*, vol. 30, p. 105474, Apr. 2020.

[2] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "Medmnist v2 – a large-scale lightweight benchmark for 2d and 3d biomedical image classification," *Scientific Data*, vol. 10, Jan. 2023.

[3] R. Liu, W. Dai, T. Wu, M. Wang, S. Wan, and J. Liu, "Aimic: Deep learning for microscopic image classification," *Computer Methods and Programs in Biomedicine*, vol. 226, p. 107162, Nov. 2022.